Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3
По дисциплине : «Модели решения задач в ИС»
По теме : «Бинарная классификация с
использованием кросс-энтропии»

Выполнил:
студент 3 курса
группы ИИ-26
Заруцкий В.Я.
Проверил:
Адренко К.В.

Брест 2026

Цель работы: Изучить принципы бинарной классификации с вероятностной точки зрения. Реализовать однослойную нейронную сеть (логистический регрессор) с **сигмоидной** функцией активации и функцией потерь в виде **бинарной кросс-энтропии** (Binary Cross-Entropy, BCE). Провести **сравнительный анализ** скорости сходимости и качества классификации с моделями, использующими среднеквадратичную ошибку (ЛР №1) и адаптивный шаг обучения (ЛР №2).

Вариант 4:

x_1	x_2	e
4	1	1
-4	1	1
4	-1	1
-4	-1	0

Ход работы :

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt

try:
    import seaborn as sns
    sns.set_theme(style='whitegrid')
except Exception:
    plt.style.use('ggplot')

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

def safe_log(x, eps=1e-12):
    return np.log(np.clip(x, eps, 1.0 - eps))

def load_custom_dataset():
    data = [
        [4, 1, 1],
        [-4, 1, 1],
        [4, -1, 1],
        [-4, -1, 0]
    ]
    X = np.array([[p[0], p[1]] for p in data], dtype=float)
    y01 = np.array([p[2] for p in data], dtype=int)
    y_pm = np.where(y01 == 0, -1, 1)
    return X, y01, y_pm

class LinearBinary:
    def __init__(self, input_size, init_lr=0.01):
        self.w = np.zeros(input_size, dtype=float)
        self.b = 0.0
        self.lr = init_lr if init_lr is not None else 0.0

    def raw(self, x):
        return np.dot(self.w, x) + self.b

    def predict_prob(self, x):
        return sigmoid(self.raw(x))

    def predict_label_bce(self, x, threshold=0.5):
        return 1 if self.predict_prob(x) >= threshold else 0

    def predict_label_mse(self, x):
        return 1 if self.raw(x) >= 0 else -1

    def decision_boundary(self, x_vals):
        if abs(self.w[1]) < 1e-12:
            return np.full_like(x_vals, np.nan)
        return (-(self.w[0] * x_vals + self.b) / self.w[1])

def should_print_epoch(epoch_index):
    p = epoch_index + 1
    return (p <= 10) or (p % 10 == 0)

def train_fixed_mse(model, X, y_pm, epochs=100, E_eps=1e-6, shuffle=True):
    n = X.shape[0]
    E_history = []
```

```
    for epoch in range(epochs):
        idx = np.random.permutation(n) if shuffle else np.arange(n)
        E = 0.0
        for i in idx:
            xi = X[i]
            yi = y_pm[i]
            raw = model.raw(xi)
            err = yi - raw
            model.w += model.lr * err * xi
            model.b += model.lr * err
            E += 0.5 * err**2
        E_history.append(E)
        if should_print_epoch(epoch):
            print(f"[MSE fixed] Epoch {epoch+1:3d}: E_sum = {E:.8f}")
        if E <= E_eps:
            break
    return E_history

def train_adaptive_mse(model, X, y_pm, epochs=100, E_eps=1e-6, shuffle=True):
    n = X.shape[0]
    E_history = []
    for epoch in range(epochs):
        idx = np.random.permutation(n) if shuffle else np.arange(n)
        E = 0.0
        for i in idx:
            xi = X[i]
            yi = y_pm[i]
            raw = model.raw(xi)
            err = yi - raw
            denom = 1.0 + np.dot(xi, xi)
            alpha_t = 1.0 / denom
            model.w += alpha_t * err * xi
            model.b += alpha_t * err
            E += 0.5 * err**2
        E_history.append(E)
        if should_print_epoch(epoch):
            print(f"[MSE adapt] Epoch {epoch+1:3d}: E_sum = {E:.8f}")
        if E <= E_eps:
            break
    return E_history

def train_fixed_bce(model, X, y01, epochs=100, E_eps=1e-6, shuffle=True):
    n = X.shape[0]
    E_history = []
    for epoch in range(epochs):
        idx = np.random.permutation(n) if shuffle else np.arange(n)
        E = 0.0
        for i in idx:
            xi = X[i]
            yi = y01[i]
            raw = model.raw(xi)
            y_hat = sigmoid(raw)
            grad = (y_hat - yi)
            model.w -= model.lr * grad * xi
            model.b -= model.lr * grad
            E += -(yi * safe_log(y_hat) + (1 - yi) * safe_log(1 - y_hat))
        E_history.append(E)
        if should_print_epoch(epoch):
            print(f"[BCE fixed] Epoch {epoch+1:3d}: E_sum = {E:.8f}")
        if E <= E_eps:
            break
    return E_history

def train_adaptive_bce(model, X, y01, epochs=100, E_eps=1e-6, shuffle=True):
    n = X.shape[0]
    E_history = []
    for epoch in range(epochs):
        idx = np.random.permutation(n) if shuffle else np.arange(n)
        E = 0.0
        for i in idx:
            xi = X[i]
            yi = y01[i]
            raw = model.raw(xi)
            y_hat = sigmoid(raw)
            grad = (y_hat - yi)
            denom = 1.0 + np.dot(xi, xi)
            alpha_t = 1.0 / denom
            model.w -= alpha_t * grad * xi
            model.b -= alpha_t * grad
            E += -(yi * safe_log(y_hat) + (1 - yi) * safe_log(1 - y_hat))
        E_history.append(E)
        if should_print_epoch(epoch):
            print(f"[BCE adapt] Epoch {epoch+1:3d}: E_sum = {E:.8f}")
        if E <= E_eps:
            break
    return E_history
```

```
def plot_convergence(histories, labels, colors, linestyles, markers):
    plt.figure(figsize=(10,6), dpi=120)
    ax = plt.gca()
    ax.set_prop_cycle(None)
    for E, lab, col, ls, mk in zip(histories, labels, colors, linestyles, markers):
        if len(E) == 0:
            continue
        epochs = np.arange(1, len(E)+1)
        plt.plot(epochs, E, color=col, linestyle=ls, marker=mk, markersize=6, linewidth=1.8, label=lab)
        plt.scatter(epochs[-1], E[-1], color=col, edgecolor='k', zorder=5)
        plt.annotate(f"{E[-1]:.3e}\n(p={epochs[-1]})",
                     xy=(epochs[-1], E[-1]),
                     xytext=(8, -18),
                     textcoords='offset points',
                     fontsize=9,
                     bbox=dict(boxstyle="round,pad=0.3", fc="white", alpha=0.8))
    plt.xlabel('Эпоха p', fontsize=12)
    plt.ylabel('E_sum(p)', fontsize=12)
    plt.title('Сходимость: MSE vs BCE, фиксированный vs адаптивный шаг', fontsize=14)
    plt.legend(title='Метод', fontsize=10)
    plt.grid(True, alpha=0.6)
    plt.tight_layout()
    plt.show()

def plot_decision_regions(models, model_names, X, y01, resolution=200):
    x_min, x_max = X[:,0].min()-1, X[:,0].max()+1
    y_min, y_max = X[:,1].min()-1, X[:,1].max()+1
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, resolution),
                         np.linspace(y_min, y_max, resolution))
    grid = np.c_[xx.ravel(), yy.ravel()]

    plt.figure(figsize=(9,7), dpi=120)
    ax = plt.gca()

    for idx, (model, name) in enumerate(zip(models, model_names)):
        if 'BCE' in name:
            Z = np.array([model.predict_prob(pt) for pt in grid])
            Zc = (Z >= 0.5).astype(int)
        else:
            Z = np.array([model.raw(pt) for pt in grid])
            Zc = (Z >= 0).astype(int)
        Zc = Zc.reshape(xx.shape)
        if idx == 0:
            ax.contourf(xx, yy, Zc, alpha=0.35, levels=[-0.5,0.5,1.5], colors=['#ffdddd','#ddeaff'])
        else:
            ax.contour(xx, yy, Zc, levels=[0.5], colors=['k'], linestyles=[':'], linewidths=1.5)

    colors = ['red' if label == 0 else 'blue' for label in y01]
    markers = ['s' if label == 0 else 'o' for label in y01]
    for xi, yi, c, m in zip(X, y01, colors, markers):
        ax.scatter(xi[0], xi[1], c=c, marker=m, edgecolors='k', s=140)

    x_vals = np.linspace(x_min, x_max, 200)
    for model, name, style in zip(models, model_names, ['r--','b-']):
        y_vals = model.decision_boundary(x_vals)
        if not np.isnan(y_vals).all():
            ax.plot(x_vals, y_vals, style, linewidth=2, label=f'{name} boundary')

    ax.set_xlabel('x1', fontsize=12)
    ax.set_ylabel('x2', fontsize=12)
    ax.set_title('Точки и области решений моделей', fontsize=14)
    ax.legend()
    ax.set_xlim(x_min, x_max)
    ax.set_ylim(y_min, y_max)
    ax.set_aspect('equal', adjustable='box')
    plt.grid(True, alpha=0.6)
    plt.tight_layout()
    plt.show()

def run_experiments(max_epochs=200, E_eps=1e-6, fixed_lr=0.05, seed=42):
    np.random.seed(seed)
    X, y01, y_pm = load_custom_dataset()

    m_mse_fixed = LinearBinary(input_size=2, init_lr=fixed_lr)
    m_mse_adapt = LinearBinary(input_size=2, init_lr=None)
    m_bce_fixed = LinearBinary(input_size=2, init_lr=fixed_lr)
    m_bce_adapt = LinearBinary(input_size=2, init_lr=None)

    E_mse_fixed = train_fixed_mse(m_mse_fixed, X, y_pm, epochs=max_epochs, E_eps=E_eps, shuffle=True)
    E_mse_adapt = train_adaptive_mse(m_mse_adapt, X, y_pm, epochs=max_epochs, E_eps=E_eps, shuffle=True)
    E_bce_fixed = train_fixed_bce(m_bce_fixed, X, y01, epochs=max_epochs, E_eps=E_eps, shuffle=True)
    E_bce_adapt = train_adaptive_bce(m_bce_adapt, X, y01, epochs=max_epochs, E_eps=E_eps, shuffle=True)

    histories = [E_mse_fixed, E_mse_adapt, E_bce_fixed, E_bce_adapt]
    labels = ['MSE + fixed', 'MSE + adaptive', 'BCE + fixed', 'BCE + adaptive']
```

```
    colors = ['darkred', 'orangered', 'navy', 'royalblue']
    linestyles = ['-', '--', '-', '--']
    markers = ['o', 's', 'o', 's']
    plot_convergence(histories, labels, colors, linestyles, markers)

    plot_decision_regions([m_bce_adapt, m_mse_fixed], ['BCE adaptive', 'MSE fixed'], X, y01)

    new_point = np.array([0.0, 0.0])
    prob = m_bce_adapt.predict_prob(new_point)
    cls = 1 if prob >= 0.5 else 0
    print(f"New point {new_point} -> BCE adaptive prob = {prob:.4f}, class = {cls}")

    preds_mse_fixed = np.array([m_mse_fixed.predict_label_mse(x) for x in X])
    acc_mse_fixed = np.mean(preds_mse_fixed == y_pm) * 100
    preds_mse_adapt = np.array([m_mse_adapt.predict_label_mse(x) for x in X])
    acc_mse_adapt = np.mean(preds_mse_adapt == y_pm) * 100

    preds_bce_fixed = np.array([m_bce_fixed.predict_label_bce(x) for x in X])
    acc_bce_fixed = np.mean(preds_bce_fixed == y01) * 100
    preds_bce_adapt = np.array([m_bce_adapt.predict_label_bce(x) for x in X])
    acc_bce_adapt = np.mean(preds_bce_adapt == y01) * 100

    print("\n--- Результаты на обучающей выборке ---")
    print(f"MSE fixed:   weights = {m_mse_fixed.w}, bias = {m_mse_fixed.b}, accuracy = {acc_mse_fixed:.1f}%")
    print(f"MSE adapt:   weights = {m_mse_adapt.w}, bias = {m_mse_adapt.b}, accuracy = {acc_mse_adapt:.1f}%")
    print(f"BCE fixed:   weights = {m_bce_fixed.w}, bias = {m_bce_fixed.b}, accuracy = {acc_bce_fixed:.1f}%")
    print(f"BCE adapt:   weights = {m_bce_adapt.w}, bias = {m_bce_adapt.b}, accuracy = {acc_bce_adapt:.1f}%")

    return {
        'models': {
            'mse_fixed': m_mse_fixed,
            'mse_adapt': m_mse_adapt,
            'bce_fixed': m_bce_fixed,
            'bce_adapt': m_bce_adapt
        },
        'histories': {
            'mse_fixed': E_mse_fixed,
            'mse_adapt': E_mse_adapt,
            'bce_fixed': E_bce_fixed,
            'bce_adapt': E_bce_adapt
        }
    }

if __name__ == "__main__":
    results = run_experiments(max_epochs=100, E_eps=1e-6, fixed_lr=0.05, seed=1)
```

Результат тестирования:

```
[MSE fixed] Epoch   1: E_sum = 2.56987200
[MSE fixed] Epoch   2: E_sum = 3.32767534
[MSE fixed] Epoch   3: E_sum = 0.93511221
[MSE fixed] Epoch   4: E_sum = 1.76123078
[MSE fixed] Epoch   5: E_sum = 1.60065975
[MSE fixed] Epoch   6: E_sum = 2.08194412
[MSE fixed] Epoch   7: E_sum = 1.72799688
[MSE fixed] Epoch   8: E_sum = 1.67258581
[MSE fixed] Epoch   9: E_sum = 1.03240379
[MSE fixed] Epoch  10: E_sum = 1.46466425
[MSE fixed] Epoch  20: E_sum = 0.47708432
[MSE fixed] Epoch  30: E_sum = 0.53755184
[MSE fixed] Epoch  40: E_sum = 1.76850721
[MSE fixed] Epoch  50: E_sum = 1.07610439
[MSE fixed] Epoch  60: E_sum = 1.70920039
[MSE fixed] Epoch  70: E_sum = 1.30404230
[MSE fixed] Epoch  80: E_sum = 1.44566317
[MSE fixed] Epoch  90: E_sum = 1.07576336
[MSE fixed] Epoch 100: E_sum = 1.62604783
[MSE adapt] Epoch   1: E_sum = 3.69954708
[MSE adapt] Epoch   2: E_sum = 2.72026550
[MSE adapt] Epoch   3: E_sum = 2.09343229
[MSE adapt] Epoch   4: E_sum = 1.54941081
[MSE adapt] Epoch   5: E_sum = 1.86076032
[MSE adapt] Epoch   6: E_sum = 2.14531525
[MSE adapt] Epoch   7: E_sum = 1.75961734
[MSE adapt] Epoch   8: E_sum = 1.19791233
```

```
[MSE adapt] Epoch   9: E_sum = 1.17383377
[MSE adapt] Epoch  10: E_sum = 1.30399502
[MSE adapt] Epoch  20: E_sum = 1.72614164
[MSE adapt] Epoch  30: E_sum = 1.56965099
[MSE adapt] Epoch  40: E_sum = 1.18873997
[MSE adapt] Epoch  50: E_sum = 1.58865401
[MSE adapt] Epoch  60: E_sum = 0.48778226
[MSE adapt] Epoch  70: E_sum = 1.90263025
[MSE adapt] Epoch  80: E_sum = 1.16041682
[MSE adapt] Epoch  90: E_sum = 1.50815847
[MSE adapt] Epoch 100: E_sum = 1.66019470
[BCE fixed] Epoch   1: E_sum = 2.75261018
[BCE fixed] Epoch   2: E_sum = 2.47383488
[BCE fixed] Epoch   3: E_sum = 2.29902093
[BCE fixed] Epoch   4: E_sum = 2.23954347
[BCE fixed] Epoch   5: E_sum = 2.11924610
[BCE fixed] Epoch   6: E_sum = 2.05534516
[BCE fixed] Epoch   7: E_sum = 1.93641802
[BCE fixed] Epoch   8: E_sum = 1.87336066
[BCE fixed] Epoch   9: E_sum = 1.81081133
[BCE fixed] Epoch  10: E_sum = 1.76882356
[BCE fixed] Epoch  20: E_sum = 1.28393947
[BCE fixed] Epoch  30: E_sum = 1.00149586
[BCE fixed] Epoch  40: E_sum = 0.82693746
[BCE fixed] Epoch  50: E_sum = 0.70098460
[BCE fixed] Epoch  60: E_sum = 0.61245426
[BCE fixed] Epoch  70: E_sum = 0.53860385
[BCE fixed] Epoch  80: E_sum = 0.48120239
[BCE fixed] Epoch  90: E_sum = 0.43439847
[BCE fixed] Epoch 100: E_sum = 0.39507402
[BCE adapt] Epoch   1: E_sum = 2.75234922
[BCE adapt] Epoch   2: E_sum = 2.47721852
[BCE adapt] Epoch   3: E_sum = 2.39691608
[BCE adapt] Epoch   4: E_sum = 2.26782199
[BCE adapt] Epoch   5: E_sum = 2.12136218
[BCE adapt] Epoch   6: E_sum = 2.06410393
[BCE adapt] Epoch   7: E_sum = 1.94051118
[BCE adapt] Epoch   8: E_sum = 1.82761113
[BCE adapt] Epoch   9: E_sum = 1.80525891
[BCE adapt] Epoch  10: E_sum = 1.70137790
[BCE adapt] Epoch  20: E_sum = 1.23743423
[BCE adapt] Epoch  30: E_sum = 0.95842944
[BCE adapt] Epoch  40: E_sum = 0.77875203
[BCE adapt] Epoch  50: E_sum = 0.64951851
[BCE adapt] Epoch  60: E_sum = 0.56500259
[BCE adapt] Epoch  70: E_sum = 0.49244540
[BCE adapt] Epoch  80: E_sum = 0.44000658
[BCE adapt] Epoch  90: E_sum = 0.39539109
[BCE adapt] Epoch 100: E_sum = 0.36064525

--- Результаты на обучающей выборке ---
MSE fixed:   weights = [0.02471032 0.4980456 ], bias = 0.4996734188794888, accuracy = 100.0%
MSE adapt:   weights = [0.21461361 0.4789326 ], bias = 0.33738706233946003, accuracy = 75.0%
BCE fixed:   weights = [0.58436084 1.95875713], bias = 1.9619029729475523, accuracy = 100.0%
BCE adapt:   weights = [0.59642212 2.06246866], bias = 2.063452474703488, accuracy = 100.0%
```

График с разделяющей поверхностью:

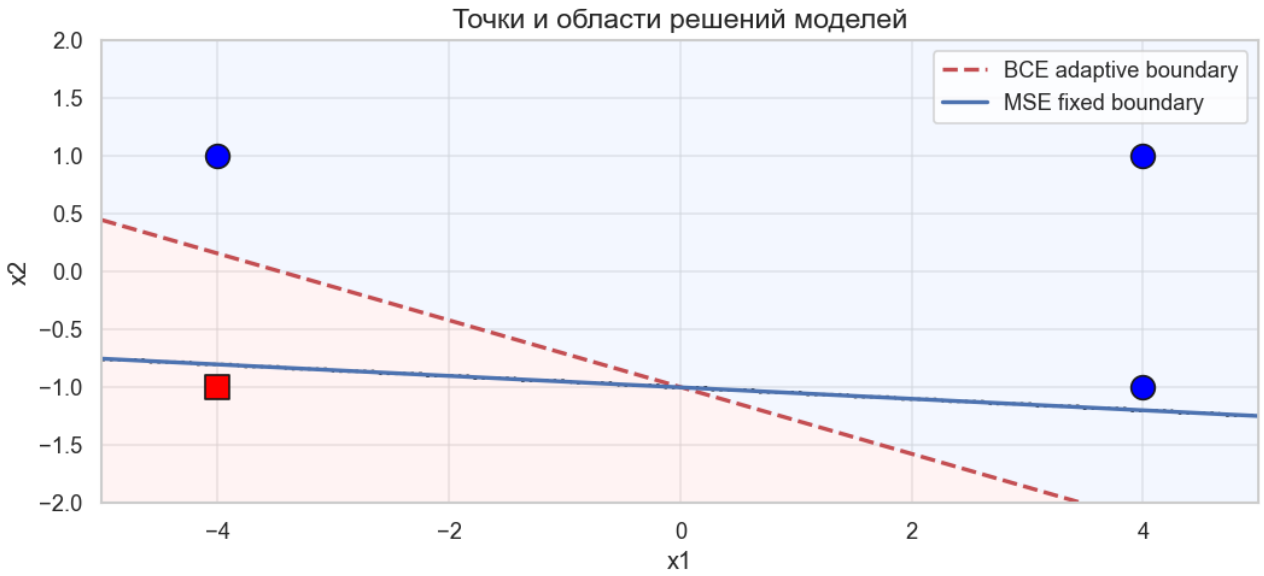
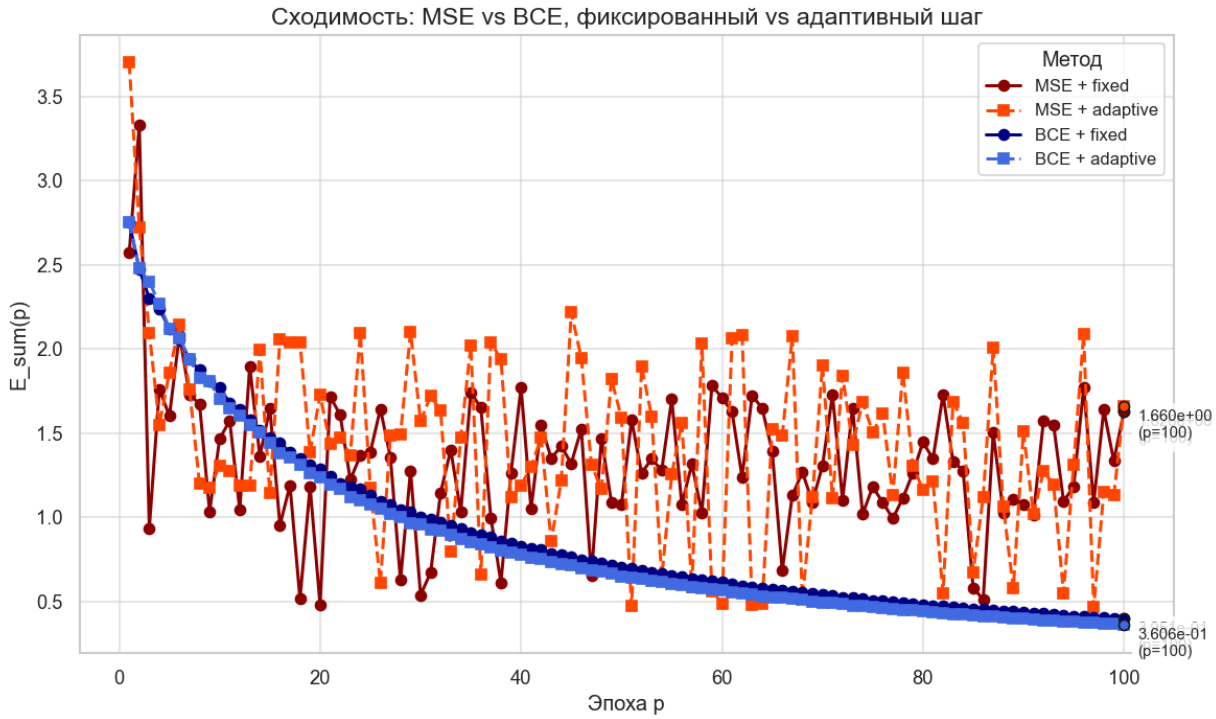


График изменения ошибки:



Вывод: Изучил принципы бинарной классификации с вероятностной точки зрения. Реализовал однослойную нейронную сеть (логистический регрессор) с сигмоидной функцией активации и функцией потерь в виде бинарной кросс энтропии (Binary Cross Entropy, BCE). Провел сравнительный анализ скорости сходимости и качества классификации с моделями, использующими среднеквадратичную ошибку (ЛР №1) и адаптивный шаг обучения (ЛР №2).